Task 1

Term	Simple meaning
API	API do the work of Communication between to program
Request	The "ask" your app sends — e.g. "give me today's weather for Pune"
Response	The "answer" that comes back — usually as JSON
Endpoint	An endpoint is just the exact web address that tells the API precisely what data you're asking for.
JSON	The text format most responses are written in — plain text organized as key-value pairs, easy for programs (and humans) to read
Third-party API	An API owned by someone else — you just use it instead of building that feature yourself

Take **Zomato**, for example.

Feature: the map showing your delivery rider's live location.

Why it's very likely a third-party API: Zomato is a food-ordering company, not a mapping company. Building an accurate, real-time map with roads, traffic, and location tracking from scratch would take years. It's far more likely they send a **request** to a mapping API (like Google Maps) with the rider's coordinates, and that API sends back a **response** (JSON with map tiles, route data) which Zomato displays inside its own app.

Approch	Strength	Weakness
Using a third-party API	Fast to add, built by experts, proven at scale	You depend on someone else's service being up and priced fairly
Building your own feature from scratch	Full control over exactly how it works	Takes huge time, money, and expertise to build well

• If a third-party API you depend on suddenly goes down, what happens to your app? Whose fault is it, and who fixes it?

 Zomato's app itself is still running fine

 But the **map feature specifically breaks** — you might see a blank space, a loading spinner that never stops, or an error like "Unable to load map"

 Other features (like browsing restaurants, or the food menu) might still work completely normally, because those probably depend on *different* services

• Why do you think almost no company builds 100% of its own features from scratch anymore?

That would take:

- Thousands of engineers
- Years of work
- Billions of dollars
- Constant updates (roads change, new buildings come up)

Meanwhile, Google already spent all that effort building it — and made it available for other companies to *use* through an API, usually for a fee.

Task 2

• REST API — a request-then-response style of communication, one question at a time

• GET / POST / PUT / DELETE — the four basic types of requests — read, create, update, delete

• Status code — a number the server sends back saying what happened (200 = success, 404 = not found)

• WebSocket — a connection that stays open so either side can send a message anytime

• Polling — repeatedly asking 'anything new?' every few seconds using REST, instead of using WebSocket

• Real-time — updates that appear instantly, without the user refreshing or re-asking

In a food delivery app, which feature do you think uses REST, and which uses WebSocket? Explain your reasoning for both, using the app's actual screens as your example.

A) In food delivery app which is the use for order & search use the REST API
& For tracking the Rider WebSocket is use.

• Why would using WebSocket for EVERY feature in an app (even the ones that barely change) actually be a bad idea?

A) Yes Using WebSocket all the time for unnecessary is bad idea because WebSocket keep continuous open connection with Server . its is use for real time update. If our feature is not work on real time update its is unnecessary to use WebSocket.

• Can you think of a feature where using REST + polling instead of WebSocket might be 'good enough'? Why?

A) Yes we can use in Food Delivery app for check is order is ready or not Rest polling always send request is order ready after getting order ready we will get response Rest Polling Always send request after 5 to 10 sec.

Task 3

• Frontend — everything the user actually sees and clicks — the visible part of the app

• Backend — the part running on a server that handles data, logic, and rules — invisible to the user

• Framework — a ready structure/toolkit that makes building an app faster (e.g. React, Django)

• Runtime — the environment that actually runs your backend code (e.g. Node.js runs JavaScript on a server)

• Full stack — someone comfortable working on both frontend and backend

• Stack — the specific combination of tools/technologies a project is built with

Pick any app. Describe ONE thing you see on screen (frontend) and guess ONE thing happening behind the scenes when you tap it (backend) — explain the difference in your own words.

A) Frontend is what the user can see and interact with, while backend is the hidden part that processes the user's action and manages the data.

• Why do you think 'using the same language on frontend and backend' is considered an advantage for small teams?

A) Using the **same language on frontend and backend** is an advantage for small teams because the developers don't have to learn and manage many different languages.

• If a company already has 50 Python developers, would you recommend they switch to Node.js just because it's popular? Why or why not?

A) No there is no need to Switch because I have already have python Expert And in Node.js java script is use .

Task 4

• Table / Row / Column — how SQL organizes data — like a spreadsheet

• Document — how NoSQL (MongoDB) organizes data — like a flexible JSON object

• Schema — the fixed structure/rules a SQL table must follow

• Schema-less — NoSQL's flexibility — documents don't all have to look the same

• Foreign key — a link connecting two SQL tables together (e.g. a user_id linking Orders to Users)

• Relationship — how data in one table connects to data in another

Instagram stores structured data like usernames and follower counts, but also very flexible data like post captions and comments. Which parts do you think fit SQL better, and which fit NoSQL better? Explain your guess.

A) The username and user id ,Followers count it is a fixed datatype for that we can use SQL And for captions ,post, comment we use NOSQL Because the datatype is not fixed.

• Why might a banking app almost always prefer SQL over NoSQL?

A) Because there datatype is fixed

• Why might a fast-moving startup prototype prefer NoSQL in its very first version?

A) For Startup we will use NOSQL because we don't know the datatype Structure.

Task 5

• Container — a packaged app + everything it needs to run, so it behaves the same anywhere

• Image — the 'recipe' used to create a container • Docker — the tool used to build and run containers

• Kubernetes — a tool that manages, restarts, and scales MANY containers automatically

• Pod — the smallest unit Kubernetes manages (usually one container, sometimes a few together)

• Scaling — adding more copies of something to handle more users

Imagine a college project with just 1 backend service vs a company like Zomato with hundreds of services running at once. Which one do you think actually needs Kubernetes, and which one doesn't? Explain why.

A) **Zomato Company will need Kubernetes because there are many user which will use that file and Kubernetes scale its self the container depend on how user increase and decrease.**

• Why would adding Kubernetes to a tiny 1-service college project actually be a bad decision, not a smart one?

A) **Kubernates Have power to handel multiple user but for clg Project there is no need to use Kubernates there will be not multiuser there will be one user only .**

• What real-world problem is Docker solving that existed even BEFORE Kubernetes was invented?

A) **Docker solves the problem of making sure an application runs the same way on different computers**

Task 6

• Reverse proxy — something that sits in front of your servers and forwards requests to the right one

• Nginx — a popular, general-purpose reverse proxy / web server tool

• API Gateway — a reverse proxy built specifically to manage APIs — auth, rate limits, routing

• Kong — a popular tool built specifically to be an API Gateway

• Rate limiting — restricting how many requests one user/key can make in a given time

• Authentication (at the gateway) — checking 'who are you' before letting a request through at all

Think of a big company like Ola with dozens of internal services (driver matching, pricing, trip history). Why do you think they would NOT want every request going straight to those services with zero checks?

A) **Companies put checks in front of internal services to protect them, control access, prevent abuse, and keep the whole application reliable.**

• Why is 'one central checkpoint' usually safer than each service checking security on its own?

A) **The gateway can consistently check authentication, permissions, rate limits, and other rules before requests reach the services.**

• Can you think of a REAL situation where too many requests from one user should be blocked automatically?

A) **Central checkpoints provide consistent security, while rate limiting stops users or programs from sending an unreasonable number of requests and overwhelming or abusing a service.**

Task 7

• Authentication — proving who you are (e.g. logging in)

• Authorization — deciding what you're allowed to do once logged in

• Session — the server remembering you're logged in, usually using a cookie

• Token / JWT — a piece of proof you carry with you instead of the server remembering you

• OAuth — a way to log in using another trusted account (Google, Facebook) without sharing that password

• Stateless — the server doesn't need to 'remember' anything about you — the token carries the proof

Think of an app where you log in once and stay logged in for days, vs an app where you get logged out the moment you close it. What's different about how each might be handling your login?

A) App you logibn for days
B) Login
C) ↓
D) Server verifies you
E) ↓
F) Creates a session/token
G) ↓
H) App stores it
I) ↓
J) You can use the app for days
K) The app might use a **long-lived session or token**. When you open the app again, it can prove to the server that you already logged in.

B) App where you are logged out when you close it
C) Login
D) ↓
E) Temporary session
F) ↓
G) Close app
H) ↓
I) Session information is removed/expired
J) ↓
K) Login required again
L) It might use a **short-lived session** or deliberately avoid saving the login information after the app closes

1. **Why is asking for your Google password directly risky?**
   That app now knows your Google password. If the app is hacked, badly designed, or dishonest, your password could be exposed.

• If your app runs on 10 different servers at once, why would JWT be easier to manage than sessions?

A) Sessions often require servers to share session information, while JWTs can be verified independently by each server. This makes JWT convenient when an application is running across many servers.

Task 8

• Deployment — taking your code from your laptop and making it live for real users

• Hosting — the service/place where your live app's code actually runs

• Server — a computer (usually far away, always on) that runs your app so others can use it

• Domain — the readable address (like nsgacademy.com) that points to your server

• Uptime — how much of the time your app stays up and working, without crashing

• CI/CD — automatically testing and deploying every code change, instead of doing it by

Think about what would need to be true for an app like Zomato to NEVER go down, even at 9pm on a Saturday when millions of people order food at once. Name one thing you think they must be doing behind the scenes.

A) One important thing they must be doing is **using multiple servers and load balancing**.
B) Instead of sending every request to one server, a **load balancer** distributes requests across many servers.
C) So if one server becomes overloaded or fails, other servers can continue handling users.

1. Why don't companies manually upload files to the live app?

D) Because it is **slow, error-prone, and difficult to repeat**.
E) Instead, companies usually use **CI/CD pipelines**:

• What is one thing that could go wrong if a developer deploys code straight from their laptop with no testing step at all?

A) **Companies automate deployment because manual uploading is risky and difficult to manage. Without testing, a developer could accidentally deploy a bug that breaks an important feature for many users.**